Data Structures and Algorithms

INFORMATION
TECHNOLOGY

Objectives

- In this session, you will learn to:
  - Sort data by using quick sort
  - Search data by using linear search technique
  - Search data by using binary search technique

Sorting Data by Using Quick Sort

- Quick Sort Algorithm:
  - Is one of the most efficient sorting algorithms.
  - Is based on the divide and conquer approach.
  - Successively divides the problem into smaller parts until the problems become so small that they can be directly solved.

Implementing the Quick Sort Algorithm

- In quick sort algorithm, you:
  - Select an element from the list called as pivot.
  - Partition the list into two parts such that:
    - All the elements towards the left end of the list are smaller than the pivot.
    - All the elements towards the right end of the list are greater than the pivot.
  - Store the pivot at its correct position between the two parts of the list.
- You repeat this process for each of the two sub lists created after partitioning.
- This process continues until one element is left in each sub list.

Let us view an animation to understand the implementation of the quick sort algorithm.

Implementing the Quick Sort Algorithm (Contd.)

- The following algorithm depicts the logic of quick sort:

```
QuickSort(low,high)
  1. If (low > high):
       Return.
  2. Set pivot = arr[low].
  3. Set i = low + 1.
  4. Set j = high.
  5. Repeat step 6 until i > high or arr[i]> pivot.
     // Search for an element greater than the pivot.
```

Implementing the Quick Sort Algorithm (Contd.)

```
6. Increment i by 1.
7. Repeat step 8 until j < low or arr[j] < pivot.
   //Search for an element smaller than pivot
8. Decrement j by 1.
9. If i < j:
   // If greater element is on the left of
   //smaller element
     Swap arr[i] with arr[j].
10. If i <= j:
     Go to step 5. // Continue the search
11. If low < j:
     Swap arr[low] with arr[j].
     //Swap pivot with last element in first part of
     //the list
```

Implementing the Quick Sort Algorithm (Contd.)

```
12. QuickSort(low, j - 1). // Apply quicksort on
    //list left to pivot
13. QuickSort(j + 1, high). // Apply quicksort on
    //list right to pivot
```

Determining the Efficiency of the Quick Sort Algorithm

- The total time taken by this sorting algorithm depends on the position of the pivot value.
- If the first element is chosen as the pivot, it leads to a worst case efficiency of $O(n^2)$.
- The worst case occurs when the list is already sorted.
- In this case, the total number of comparisons is:

$$\begin{aligned}\text{Number of comparisons} &= n + (n - 1) + (n - 2) + .. + 2 + 1 \\ &= n(n - 1)/2 \\ &= O(n^2)\end{aligned}$$

Determining the Efficiency of the Quick Sort Algorithm (Contd.)

- If you select the median of all values as the pivot, the efficiency will be `O(n log n)`.This is because:
  - Partitioning the initial list places one element at its correct position and produces two sub lists of nearly equal sizes.
  - Partitioning the two sub lists places two elements at their correct positions and produces four sub lists of nearly equal sizes.
  - Partitioning the four sub lists places four elements at their correct positions and produces eight sub lists of nearly equal sizes.
- This process continues until all the elements are placed at their correct positions.
- This means that you require approximately $\log_2 n$ reductions to sort a list of n elements.
- In each reduction, there are a maximum of `n` comparisons.

Just a Minute

- Which one of the following options specifies the approximate number of reductions required to sort a list of n elements by using quick sort?
  - $\log_2 n$
  - $n \log n$
  - n^2

Just a Minute (Contd.)

- Solution:
  - $\log_2 n$

Just a Minute (Contd.)

- What is the efficiency of the quick sort algorithm when the median of the list is chosen as the pivot?
  - O(1)
  - O(log n)
  - O(n log n)
  - $O(n^2)$

Just a Minute (Contd.)

- Solution:
  - `O(n log n)`

Demo: Sorting Data by Using the Quick Sort Algorithm

- Problem Statement:
  - Write a program that stores 10 numbers in an array and sorts them by using the quick sort algorithm. In addition, the program should also calculate the number of comparisons and the number of data movements.

Demo: Sorting Data by Using the Quick Sort Algorithm (Contd.)

- Solution:
  - To sort the list of numbers by using the quick sort algorithm, you need to perform the following tasks:
    1. Write the code.
    2. Execute the code and verify the output.

Performing Linear Search

- Linear Search:
  - Is the simplest searching method.
  - Is also referred to as sequential search.
  - Involves comparing the items sequentially with the elements in the list.

Implementing Linear Search

- The linear search would begin by comparing the required element with the first element in the list.
- If the values do not match:
  - The required element is compared with the second element in the list.
- If the values still do not match:
  - The required element is compared with the third element in the list.
- This process continues, until:
  - The required element is found or the end of the list is reached.

Let us view an animation to understand the implementation of the linear search.

Data Structures and Algorithms

Implementing Linear Search (Contd.)

- The following algorithm depicts the logic to search an employee ID in an array by using linear search:

```
1. Read the employee ID to be searched.
2. Set i = 0.
3. Repeat step 4 until i = n or arr[i] = employee ID.
4. Increment i by 1.
5. If i = n:
      Display "Not Found".
   Else:
      Display "Found".
```

Determining the Efficiency of Linear Search

- The efficiency of a searching algorithm is determined by the running time of the algorithm.
- In the best case scenario:
  - The element is found at the first position in the list.
  - The number of comparisons in this case is 1.
  - The best case efficiency of linear search is therefore, `O(1)`.
- In the worst case scenario:
  - The element is found at the last position of the list or does not exists in the list.
  - The number of comparisons in this case is equal to the number of elements.
  - The worst case efficiency of linear search is therefore, `O(n)`.

Determining the Efficiency of Linear Search (Contd.)

- In the average case scenario:
  - The number of comparisons for linear search can be determined by finding the average of the number of comparisons in the best and worst case.
  - The average number of comparisons is `(n + 1)/2`.

Just a Minute

- While implementing linear search, if you find that the element is not present in an array of size `n`, how many number of comparisons would you have made to search the required element in the given list?
  - `1`
  - `n-1`
  - `n`

Data Structures and Algorithms

Just a Minute (Contd.)

- Solution:
  - n

Demo: Performing Linear Search

- Problem Statement:
  - Write a program to search a given number in an array that contains a maximum of 20 numbers by using the linear search algorithm. If there are more than one occurrences of the element to be searched, then the program should display the position of the first occurrence. The program should also display the total number of comparisons made.

Demo: Performing Linear Search (Contd.)

- Solution:
  - To implement the linear search algorithm, you need to perform the following tasks:
    1. Write the code.
    2. Execute the code and verify the output.

Performing Binary Search

- Binary Search Algorithm:
  - Is used for searching large lists.
  - Searches the element in very few comparisons.
  - Can be used only if the list to be searched is sorted.

Implementing Binary Search

- Consider an example where you have to search the name, Steve, in a telephone directory that is sorted alphabetically.
- To search the name, Steve, by using binary search algorithm:
  - You open the telephone directory at the middle to determine which half contains the name.
  - Open that half at the middle to determine which quarter of the directory contains the name.
- Repeat this process until the name, Steve, is not found.
- Binary search reduces the number of pages to be searched by half each time.

Let us view an animation to understand the implementation of binary search.

Implementing Binary Search (Contd.)

- The following algorithm depicts the logic to search a desired element by using binary search:

```
1. Accept the element to be searched.
2. Set lowerbound = 0.
3. Set upperbound = n - 1.
4. Set mid = (lowerbound + upperbound)/2.
5. If arr[mid] = desired element:
    a. Display "Found".
    b. Go to step 10.
6. If desired element < arr[mid]:
    Set upperbound = mid - 1.
```

Data Structures and Algorithms

Implementing Binary Search (Contd.)

```
7. If desired element > arr[mid]:
     Set lowerbound = mid + 1.
8. If lowerbound <= upperbound:
     Go to step 4.
9. Display "Not Found".
10. Exit.
```

Determining the Efficiency of Binary Search

- In binary search, with every step, the search area is reduced to half.
- In the best case scenario, the element to be search is found at the middlemost position of the list:
  - The number of comparisons, in this case, is 1.
- In the worst case scenario, the element is not found in the list:
  - After the first bisection, the search space is reduced to $n/2$ elements, where n is the number of elements in the original list.
  - After the second bisection, the search space is reduced to $n/4$ elements, that is, $n/2^2$ elements.
  - After the ith bisection, the search space is reduced to $n/2^i$ elements.

Determining the Efficiency of Binary Search (Contd.)

Suppose, after the ith bisection, the search space is reduced to one element. In this case,

$$n/2^i = 1$$

$$n = 2^i$$

$$i = \log_2 n \text{ (If } a = 2^b\text{, then } b = \log_2 a\text{)}$$

- This means that the list can be bisected in maximum $\log_2 n$ times. After each bisection, only one comparison is made. Therefore, the total number of comparisons will be $\log_2 n$. This means that the worst case efficiency of binary search is $O(\log n)$.

Just a Minute

- In _____ search algorithm, you begin at one end of the list and scan the list until the desired item is found or the end of the list is reached.

Just a Minute (Contd.)

- Solution:
  - Linear

Just a Minute (Contd.)

- To implement _____ search algorithm, the list should be sorted.

Just a Minute (Contd.)

- Solution:
  - Binary

Demo: Performing Binary Search

- Problem Statement:
  - Write a program to search a number in an array that contains a maximum of 20 elements by using binary search. Assume that the array elements are entered in the ascending order and all the array elements are unique. The program should also display the total number of comparisons made.

Demo: Performing Binary Search (Contd.)

- Solution:
  - To implement the binary search algorithm, you need to perform the following tasks:
    - Write the code.
    - Execute the code and verify the output.

Summary

- In this session, you learned that:
  - The quick sort algorithm recursively divides the list into two sub lists.
  - The worst case efficiency of the quick sort algorithm is $O(n^2)$.
  - The best case efficiency of the quick sort algorithm is $O(n \log n)$.
  - The best case of efficiency of linear search is $O(1)$, and the worst case of efficiency of linear search is $O(n)$.
  - To apply binary search algorithm, you should ensure that the list to be searched is sorted.
  - The best case of efficiency of binary search is $O(1)$, and the worst case of efficiency of binary search is $O(\log n)$.